

Polars - Parallel Data Processing in Python without Spark

Description

- Polars is a new and performant data processing module that originated in Rust and was ported to Python.
- Its design is modeled after pandas with majority of pandas-related classes and methods being identical.

Key Differences

- Polars can use multiple cores to process data whereas pandas handles everything as a single process.
- Contains an interface that is compliant to users who come from:
 - Pandas
 - Spark

Setup and Installation

- You can install `polars` via `pip`.

```
pip install polars
```

Basic Usage - Creating a DataFrame

- Classes in `polars` are modeled after `pandas`.
 - Index
 - Series
 - DataFrame
- DataFrames can either be:
 - Entirely materialized DataFrames
 - LazyFrames
 - Resulting from data scans where the data itself doesn't persist (generator-like behavior)
 - Builds a query plan, defers execution, and allows optimizations before it becomes a final DataFrame loaded in-memory.

```
import polars as pl

df = pl.DataFrame({
    "id" : [1,2,3,4],
    "value" : [10, 20, 30, 40]
})
```

- Read or scan from a CSV

...

```
csv_file = 'example.csv'
```

```
# Fully materialized DataFrame
df = pl.read_csv(csv_file)
```

```
# LazyFrame
df = pl.scan_csv(csv_file)
```

- Supports various formats similar to pandas
 - Excel
 - Parquet
 - JSON
 - ...

SQL Context

- .sql() method

...

```
# self is the name of the DataFrame as a SQL table
filtered = df.sql("SELECT id, value FROM self WHERE value >= 30")
print(filtered)
```

- SQLContext class
 - Key in the frames dictionary represents name of the table when executing SQL query on the DataFrame.

```

ctx = pl.SQLContext(frames={"main" : df})

# DataFrame
result = ctx.execute(
    """
    SELECT
        id, (value * 2) as value_squared
    FROM main
    WHERE id % 2 = 0
    """,
    eager=True
)

print(result)

```

Processing Methods

Filtering

- Filtering is possible using the `.filter()` method from both `LazyFrame` and `DataFrame` objects in polars.
 - We use `pl.col()` to refer to a specific column in the `DataFrame`.

```

import polars as pl

data = {
    'col_1' : [1,2,3,4,5],
    'col_2' : [6,7,8,9,10]
}

df = pl.DataFrame(data)

# Get the values from column 1 that are greater than 3
df = df.filter(pl.col('col_1') > 3)

print(df)

```

- You can also perform compound filtering by separating operators with `&` or `|` similar to pandas.
- `pl.col()` also supports methods such as:
 - `is_in()`

- `is_between()`

```
country_list = ['usa', 'uae']
```

```
data = {  
    'col_1' : ['charlie', 'chaplin', 'heineken'],  
    'col_2' : ['usa', 'uae', 'de']  
}
```

```
df = pl.DataFrame(data)
```

```
# Use .is_in()  
result_df = df.filter(pl.col('col_2').is_in(country_list))  
  
print(result_df)
```

Joins

```
import polars as pl  
  
df1 = pl.DataFrame({"id": [1, 2, 3], "name": ["Alice", "Bob", "Charlie"]})  
df2 = pl.DataFrame({"id": [2, 3, 4], "age": [25, 30, 35]})  
  
# Inner join on id  
result = df1.join(df2, on="id", how="inner")  
  
# Left join: keep all rows from df1, match where possible in df2  
result2 = df1.join(df2, on="id", how="left")
```

Group By

- Aggregations are handled via `.agg()`

```
df.group_by(  
    by="col1"                # or list of cols or expressions  
    , maintain_order=True    # optional: preserve input row order per group  
) .agg(  
    pl.col("value").sum().alias("value_sum"),  
    pl.mean("other_col").alias("other_mean"),  
    # another expression  
)
```

Handling JSON

- `.json_normalize()` exists

```
import polars as pl

data = [
    {
        "id": 1,
        "name": "Cole Volk",
        "fitness": {"height": 180, "weight": 85},
    },
    {
        "id": 2,
        "name": "Faye Raker",
        "fitness": {"height": 155, "weight": 58},
    },
    {
        "name": "Mark Reg",
        "fitness": {"height": 170, "weight": 78},
    },
]

df = pl.json_normalize(data, max_level=1)
print(df)
```

- Turn arrays of values into separate rows via `.explode()`

```
import polars as pl

df = pl.DataFrame({
    "letters": ["a", "a", "b", "c"],
    "numbers": [[1], [2,3], [4,5], [6,7,8]],
})

print(df)

exploded = df.explode("numbers")
print(exploded)
```

User Defined Functions

- Element-wise transformations
 - `map_elements()`

```
import math
import polars as pl

df = pl.DataFrame({
    "values": [10, 7, 1, 23]
})

def my_log(x):
    return math.log(x)

# Apply UDF to each element
result = df.with_columns(
    pl.col("values")
        .map_elements(my_log, return_dtype=pl.Float64)
        .alias("log_value")
)

print(result)
```

- Series-wide transformation
 - `map_batches()`

```

import polars as pl

df = pl.DataFrame({
    "keys": ["a","a","b","b"],
    "values": [10, 7, 1, 23]
})

def diff_from_mean(series: pl.Series) -> pl.Series:
    total = series.sum()
    mean = total / len(series)
    return series.apply(lambda v: v - mean)

result = df.select(
    pl.col("values").map_batches(diff_from_mean, return_dtype=pl.Float64)
)

print(result)

```

- Combining multiple columns
 - Use `pl.struct()`

```

import polars as pl

df3 = pl.DataFrame({
    "values_1": [1, 2, 3],
    "values_2": [10, 20, 30]
})

def add_columns(arr1, arr2):
    return arr1 + arr2

out = df3.select(
    pl.struct(["values_1", "values_2"])
    .map_batches(
        lambda s: s.struct.field("values_1") + s.struct.field("values_2"),
        return_dtype=pl.Int64
    ).alias("sum_vals")
)

print(out)

```